SQL - Funkcje okna (Window functions)
Lab 2

Imiona i nazwiska: Natalia Bratek, Jakub Karczewski

Celem ćwiczenia jest zapoznanie się z działaniem funkcji okna (window functions) w SQL, analiza wydajności zapytań i porównanie z rozwiązaniami przy wykorzystaniu "tradycyjnych" konstrukcji SQL

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

> Wyniki:

```
-- ...
```

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- MS SQL Server - wersja 2019, 2022
- PostgreSQL - wersja 15/16/17
- SQLite
- Narzędzia do komunikacji z bazą danych
  - SSMS - Microsoft SQL Managment Studio
  - DtataGrip lub DBeaver
- Przykładowa baza Northwind/Northwind3
  - W wersji dla każdego z wymienionych serwerów

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Dokumentacja/Literatura

- Kathi Kellenberger, Clayton Groom, Ed Pollack, Expert T-SQL Window Functions in SQL Server 2019, Apres 2019
- Itzik Ben-Gan, T-SQL Window Functions: For Data Analysis and Beyond, Microsoft 2020

- Kilka linków do materiałów które mogą być pomocne - https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql?view=sql-server-ver16
    - https://www.sqlservertutorial.net/sql-server-window-functions/
    - https://www.sqlshack.com/use-window-functions-sql-server/
    - https://www.postgresql.org/docs/current/tutorial-window.html
    - https://www.postgresqltutorial.com/postgresql-window-function/
    - https://www.sqlite.org/windowfunctions.html
    - https://www.sqlitetutorial.net/sqlite-window-functions/
- W razie potrzeby - opis Ikonek używanych w graficznej prezentacji planu zapytania w SSMS jest tutaj:
    - https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference

Przygotowanie

Uruchom SSMS - Skonfiguruj połączenie z bazą Northwind na lokalnym serwerze MS SQL

Uruchom DataGrip (lub Dbeaver)

- Skonfiguruj połączenia z bazą Northwind3
    - na lokalnym serwerze MS SQL
    - na lokalnym serwerze PostgreSQL
    - z lokalną bazą SQLite

Można też skorzystać z innych narzędzi klienckich (wg własnego uznania)

Oryginalna baza Northwind jest bardzo mała. Warto zaobserwować działanie na nieco większym zbiorze danych.

Korzystamy ze "zmodyfikowanej wersji" bazy northwind

Baza Northwind3 zawiera dodatkową tabelę product_history

- 2,2 mln wierszy

Bazę Northwind3 można pobrać z moodle (zakładka - Backupy baz danych)

Zadanie 1

Funkcje rankingu, `row_number()`, `rank()`, `dense_rank()`

```
select productid, productname, unitprice, categoryid,
    row_number() over(partition by categoryid order by unitprice desc) as rowno,
    rank() over(partition by categoryid order by unitprice desc) as rankprice,
    dense_rank() over(partition by categoryid order by unitprice desc) as
denserankprice
from products;
```

Wykonaj polecenie, zaobserwuj wynik. Porównaj funkcje row_number(), rank(), dense_rank(). Skomentuj wyniki.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

> Wyniki:

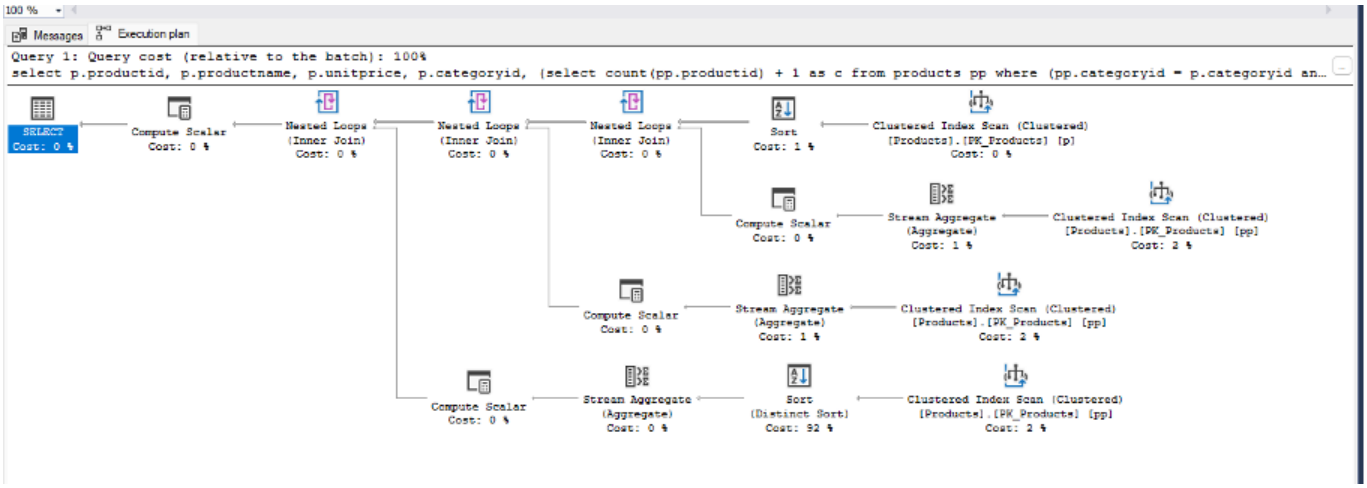
Funkcja row_number() nadaje każdemu wierszowi kolejny numer, nawet jeśli wartości są takie same. rank() daje tę samą pozycję dla takich samych wartości, ale potem są przerwy w numeracji. dense_rank() działa podobnie do rank(), tylko że nie ma tych przerw i numeracja idzie po kolei.

```
select p.productid,
       p.productname,
       p.unitprice,
       p.categoryid,
       (select count(pp.productid) + 1 as c
        from products pp
        where (pp.categoryid = p.categoryid and pp.unitprice > p.unitprice)
           or (pp.categoryid = p.categoryid and pp.unitprice = p.unitprice and
pp.productid < p.productid)) as rowno,
       (select count(pp.productid) + 1
        from products pp
        where pp.categoryid = p.categoryid
          and pp.unitprice > p.unitprice) as rankprice,
       (select count(t.c) + 1
        from (select distinct pp.unitprice as c
              from products pp
              where pp.categoryid = p.categoryid
                and pp.unitprice > p.unitprice) t) as denserankprice
from products p
order by p.categoryid, p.unitprice desc, p.productid
```

110 %

Results | Messages

	productid	productname	unitprice	categoryid	rowno	rankprice	denserankprice
1	38	Côte de Blaye	263,50	1	1	1	1
2	43	Ipoh Coffee	46,00	1	2	2	2
3	2	Chang	19,00	1	3	3	3
4	1	Chai	18,00	1	4	4	4
5	35	Steeleye Stout	18,00	1	5	4	4
6	39	Chartreuse verte	18,00	1	6	4	4
7	76	Lakkalikööri	18,00	1	7	4	4
8	70	Outback Lager	15,00	1	8	8	5
9	34	Sasquatch Ale	14,00	1	9	9	6
10	67	Laughing Lumberjack Lager	14,00	1	10	9	6
11	75	Rhönbräu Klosterbier	7,75	1	11	11	7
12	24	Guaraná Fantástica	4,50	1	12	12	8
13	63	Vegie-spread	43,90	2	1	1	1
14	8	Northwoods Cranberry Sa...	40,00	2	2	2	2
15	61	Sirop d'érable	28,50	2	3	3	3
16	6	Grandma's Boysenberry S...	25,00	2	4	4	4
17	4	Chef Anton's Cajun Seaso...	22,00	2	5	5	5
18	5	Chef Anton's Gumbo Mix	21,35	2	6	6	6
19	65	Louisiana Fiery Hot Peppe...	21,05	2	7	7	7



Zapytanie jest mniej wydajne, ponieważ dla każdego wiersza wykonywane są podzapytania, co widać w operacjach Nested Loop. Powoduje to wielokrotne przeszukiwanie tabeli i wydłuża czas wykonania.

Zadanie 2

Baza: Northwind, tabela product_history

Dla każdego produktu, podaj 4 najwyższe ceny tego produktu w danym roku. Zbiór wynikowy powinien zawierać:

- rok

- id produktu
- nazwę produktu
- cenę
- datę (datę uzyskania przez produkt takiej ceny)
- pozycję w rankingu
- Uporządkuj wynik wg roku, nr produktu, pozycji w rankingu

W przypadku długiego czasu wykonania ogranicz zbiór wynikowy.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSql, SQLite)

> Wyniki:

```sql
with ranking as (
    select
        YEAR([date]) as year,
        productid,
        productname,
        unitprice,
        [date],
        row_number() over (
            partition by productid, YEAR([date])
            order by unitprice desc
        ) as rank_pos
    from product_history
)
select *
from ranking
where rank_pos <= 4
order by year, productid, rank_pos;
```

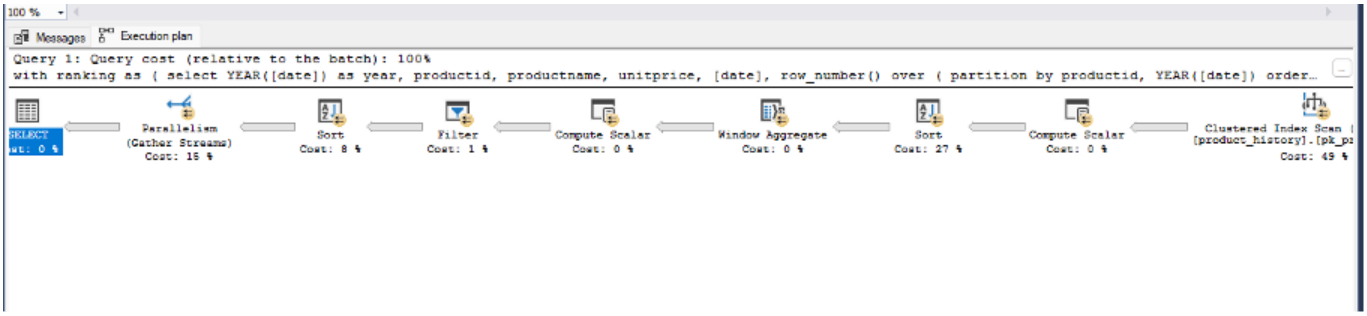
```sql
with ranking as (
    select
        YEAR([date]) as year,
        p.productid,
        p.productname,
        p.unitprice,
        p.[date],
        (
            select count(distinct p2.unitprice) + 1
            from product_history p2
            where p2.productid = p.productid
              and YEAR(p2.[date]) = YEAR(p.[date])
```

```
                and p2.unitprice > p.unitprice
            ) as rank_pos
        from product_history p
        where p.productid < 10
    )
    select *
    from ranking
    where rank_pos <= 4
    order by year, productid, rank_pos;
```

100 %

Results | Messages

	year	productid	productname	unitprice	date	rank_pos
1	1940	1	Chai	27.98	1940-07-13	1
2	1940	1	Chai	27.95	1940-11-17	2
3	1940	1	Chai	27.92	1940-06-20	3
4	1940	1	Chai	27.87	1940-06-21	4
5	1940	2	Chang	28.98	1940-07-13	1
6	1940	2	Chang	28.95	1940-11-17	2
7	1940	2	Chang	28.92	1940-06-20	3
8	1940	2	Chang	28.86	1940-06-21	4
9	1940	3	Aniseed Sy...	19.99	1940-07-13	1
10	1940	3	Aniseed Sy...	19.97	1940-11-17	2
11	1940	3	Aniseed Sy...	19.96	1940-06-20	3
12	1940	3	Aniseed Sy...	19.93	1940-06-21	4
13	1940	4	Chef Anton...	31.98	1940-07-13	1
14	1940	4	Chef Anton...	31.94	1940-11-17	2
15	1940	4	Chef Anton...	31.90	1940-06-20	3
16	1940	4	Chef Anton...	31.84	1940-06-21	4
17	1940	5	Chef Anton...	31.33	1940-07-13	1
18	1940	5	Chef Anton...	31.29	1940-11-17	2
19	1940	5	Chef Anton...	31.26	1940-06-20	3
20	1940	5	Chef Anton...	31.19	1940-06-21	4



Zapytanie z funkcją okna jest bardziej wydajne, ponieważ wykonuje obliczenia w jednym przebiegu. Wersja z podzapytaniami jest wolniejsza, bo dla każdego wiersza przeszukuje tabelę.

W SQLite zapytanie robi pełny skan tabeli, czyli sprawdza wszystkie dane. Przez to działa wolniej, szczególnie przy podzapytaniach. Funkcja okna trochę to poprawia.

Zadanie 3

Funkcje `lag()`, `lead()`

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje `lag()`, `lead()`

```
select productid, productname, categoryid, date, unitprice,
       lag(unitprice) over (partition by productid order by date)
as previousprodprice,
       lead(unitprice) over (partition by productid order by date)
as nextprodprice
from product_history
where productid = 1 and year(date) = 2022
order by date;

with t as (select productid, productname, categoryid, date, unitprice,
                  lag(unitprice) over (partition by productid
order by date) as previousprodprice,
                  lead(unitprice) over (partition by productid
order by date) as nextprodprice
           from product_history
           )
select * from t
where productid = 1 and year(date) = 2022
order by date;
```

Jak działają funkcje `lag()`, `lead()`?

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

> Wyniki: Funkcja lag() zwraca wartość z poprzedniego wiersza, a dla pierwszego wiersza wartość wynosi NULL (brak poprzedniego). Funkcja lead() zwraca wartość z następnego wiersza, a dla ostatnego wiersza wartość wynosi NULL(brak następnego).

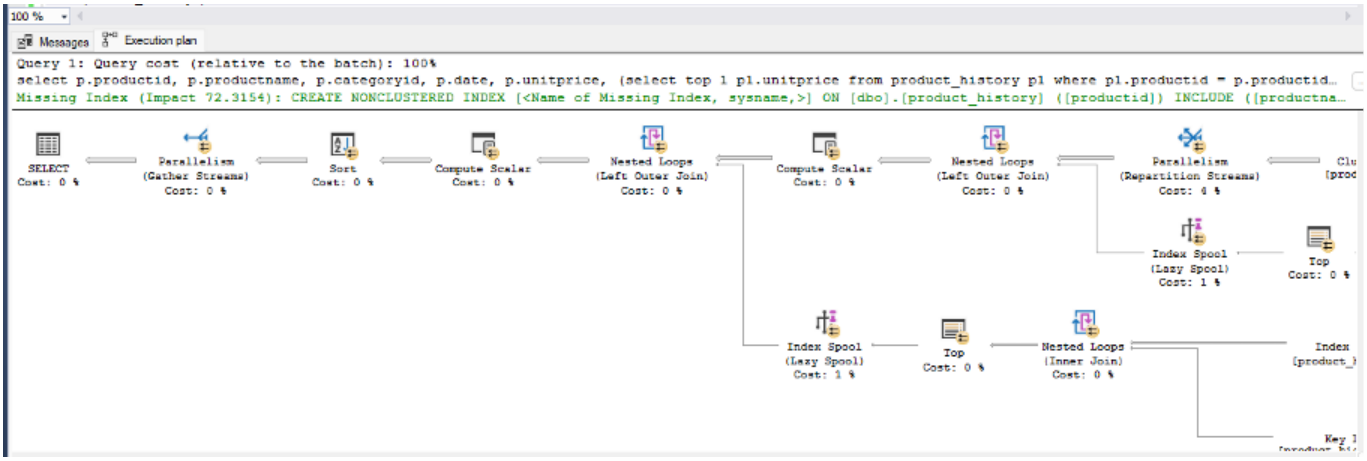
```
select p.productid,
       p.productname,
       p.categoryid,
       p.date,
       p.unitprice,

       (select top 1 p1.unitprice
        from product_history p1
        where p1.productid = p.productid
          and year(p1.date) = 2022
          and p1.date < p.date
        order by p1.date desc) as previousprodprice,

       (select top 1 p1.unitprice
```

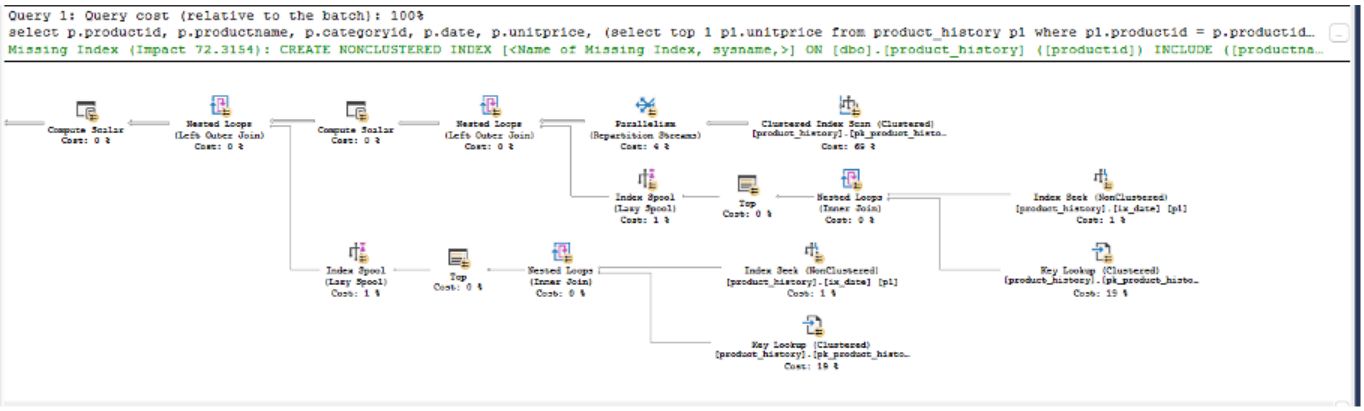
```sql
        from product_history p1
        where p1.productid = p.productid
          and year(p1.date) = 2022
          and p1.date > p.date
        order by p1.date asc) as nextprodprice

from product_history p
where p.productid = 1
  and year(p.date) = 2022
order by p.date;
```

100 %

Results | Messages

	productid	productname	categoryid	date	unitprice	previousprodprice	nextprodprice
1	1	Chai	1	2022-01-01	25.31	NULL	20.69
2	1	Chai	1	2022-01-02	20.69	25.31	18.16
3	1	Chai	1	2022-01-03	18.16	20.69	23.93
4	1	Chai	1	2022-01-04	23.93	18.16	13.69
5	1	Chai	1	2022-01-05	13.69	23.93	25.34
6	1	Chai	1	2022-01-06	25.34	13.69	17.89
7	1	Chai	1	2022-01-07	17.89	25.34	17.12
8	1	Chai	1	2022-01-08	17.12	17.89	26.73
9	1	Chai	1	2022-01-09	26.73	17.12	19.50
10	1	Chai	1	2022-01-10	19.50	26.73	10.01
11	1	Chai	1	2022-01-11	10.01	19.50	20.25
12	1	Chai	1	2022-01-12	20.25	10.01	15.67
13	1	Chai	1	2022-01-13	15.67	20.25	22.22
14	1	Chai	1	2022-01-14	22.22	15.67	16.41
15	1	Chai	1	2022-01-15	16.41	22.22	17.83
16	1	Chai	1	2022-01-16	17.83	16.41	25.14
17	1	Chai	1	2022-01-17	25.14	17.83	21.15





W zapytaniu bez funkcji okna pojawiają się operacje Nested Loop oraz dodatkowe Index Seek, co oznacza, że dla każdego wiersza wykonywane są podzapytania. Powoduje to wielokrotne przeszukiwanie tabeli i znacznie gorszą wydajność.

Zadanie 4

Baza: Northwind, tabele customers, orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- nazwę klienta, nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- nr poprzedniego zamówienia danego klienta,
- datę poprzedniego zamówienia danego klienta,
- wartość poprzedniego zamówienia danego klienta.

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

> Wyniki:

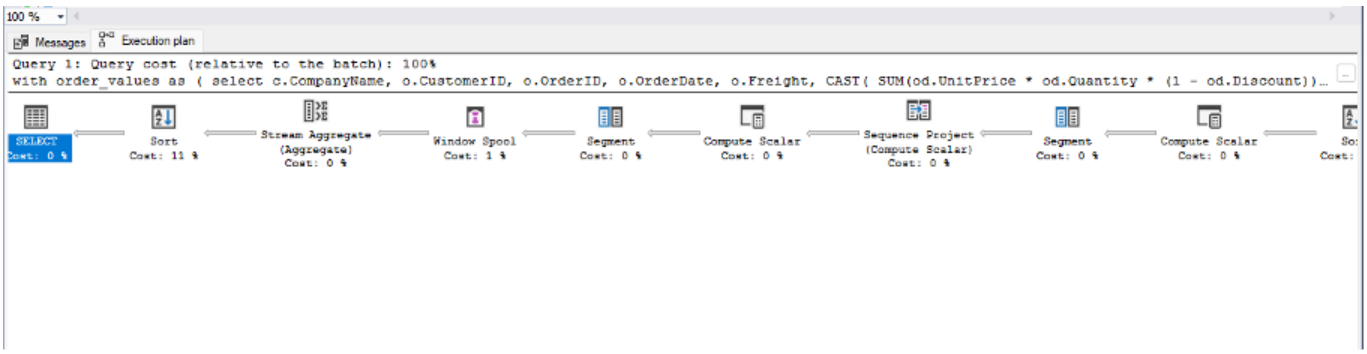
```
with order_values as (
    select
        c.CompanyName,
        o.CustomerID,
        o.OrderID,
        o.OrderDate,
        o.Freight,
        CAST(
            SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) + o.Freight
            AS decimal(10,2)
        ) as order_total
    from Orders o
    join [Order Details] od on o.OrderID = od.OrderID
    join Customers c on o.CustomerID = c.CustomerID
```

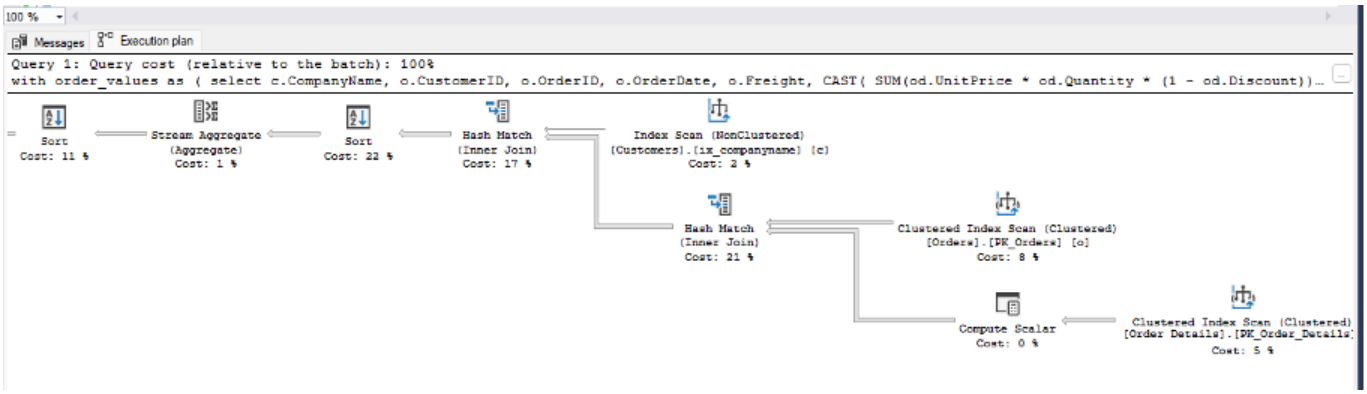
```sql
    group by
        c.CompanyName, o.CustomerID, o.OrderID, o.OrderDate, o.Freight
)

select
    CompanyName,
    OrderID,
    OrderDate,
    order_total,

    lag(OrderID) over (partition by CustomerID order by OrderDate) as
prev_order_id,
    lag(OrderDate) over (partition by CustomerID order by OrderDate) as
prev_order_date,
    lag(order_total) over (partition by CustomerID order by OrderDate) as
prev_order_value

from order_values
order by CompanyName, OrderDate;
```

100 %

Results | Messages

	CompanyName	OrderID	OrderDate	order_total	prev_order_id	prev_order_date	prev_order_value
1	Alfreds Futterkiste	10643	1997-08-25 00:00:00.000	843.96	NULL	NULL	NULL
2	Alfreds Futterkiste	10692	1997-10-03 00:00:00.000	939.02	10643	1997-08-25 00:00:00.000	843.96
3	Alfreds Futterkiste	10702	1997-10-13 00:00:00.000	353.94	10692	1997-10-03 00:00:00.000	939.02
4	Alfreds Futterkiste	10835	1998-01-15 00:00:00.000	915.33	10702	1997-10-13 00:00:00.000	353.94
5	Alfreds Futterkiste	10952	1998-03-16 00:00:00.000	511.62	10835	1998-01-15 00:00:00.000	915.33
6	Alfreds Futterkiste	11011	1998-04-09 00:00:00.000	934.71	10952	1998-03-16 00:00:00.000	511.62
7	Ana Trujillo Emparedados y helados	10308	1996-09-18 00:00:00.000	90.41	NULL	NULL	NULL
8	Ana Trujillo Emparedados y helados	10625	1997-08-08 00:00:00.000	523.65	10308	1996-09-18 00:00:00.000	90.41
9	Ana Trujillo Emparedados y helados	10759	1997-11-28 00:00:00.000	331.99	10625	1997-08-08 00:00:00.000	523.65
10	Ana Trujillo Emparedados y helados	10926	1998-03-04 00:00:00.000	554.32	10759	1997-11-28 00:00:00.000	331.99
11	Antonio Moreno Taquería	10365	1996-11-27 00:00:00.000	425.20	NULL	NULL	NULL
12	Antonio Moreno Taquería	10507	1997-04-15 00:00:00.000	796.51	10365	1996-11-27 00:00:00.000	425.20
13	Antonio Moreno Taquería	10535	1997-05-13 00:00:00.000	1956.49	10507	1997-04-15 00:00:00.000	796.51
14	Antonio Moreno Taquería	10573	1997-06-19 00:00:00.000	2166.84	10535	1997-05-13 00:00:00.000	1956.49
15	Antonio Moreno Taquería	10677	1997-09-22 00:00:00.000	817.40	10573	1997-06-19 00:00:00.000	2166.84
16	Antonio Moreno Taquería	10682	1997-09-25 00:00:00.000	411.63	10677	1997-09-22 00:00:00.000	817.40
17	Antonio Moreno Taquería	10856	1998-01-28 00:00:00.000	718.43	10682	1997-09-25 00:00:00.000	411.63
18	Around the Horn	10355	1996-11-15 00:00:00.000	521.95	NULL	NULL	NULL
19	Around the Horn	10383	1996-12-16 00:00:00.000	933.24	10355	1996-11-15 00:00:00.000	521.95
20	Around the Horn	10453	1997-02-21 00:00:00.000	422.06	10383	1996-12-16 00:00:00.000	933.24





Zapytanie najpierw łączy tabele i liczy wartość zamówień, a potem używa funkcji lag(). W planie widać sortowanie i joiny, które mają największy koszt. Całość jest wydajna, bo nie ma podzapytań dla każdego wiersza.

Zadanie 5

Funkcje `first_value()`, `last_value()`

Baza: Northwind, tabele customers, orders, order details

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje `first_value()`, `last_value()`.

Skomentuj uzyskane wyniki. Czy funkcja `first_value` pokazuje w tym przypadku najdroższy produkt w danej kategorii, czy funkcja `last_value()` pokazuje najtańszy produkt?

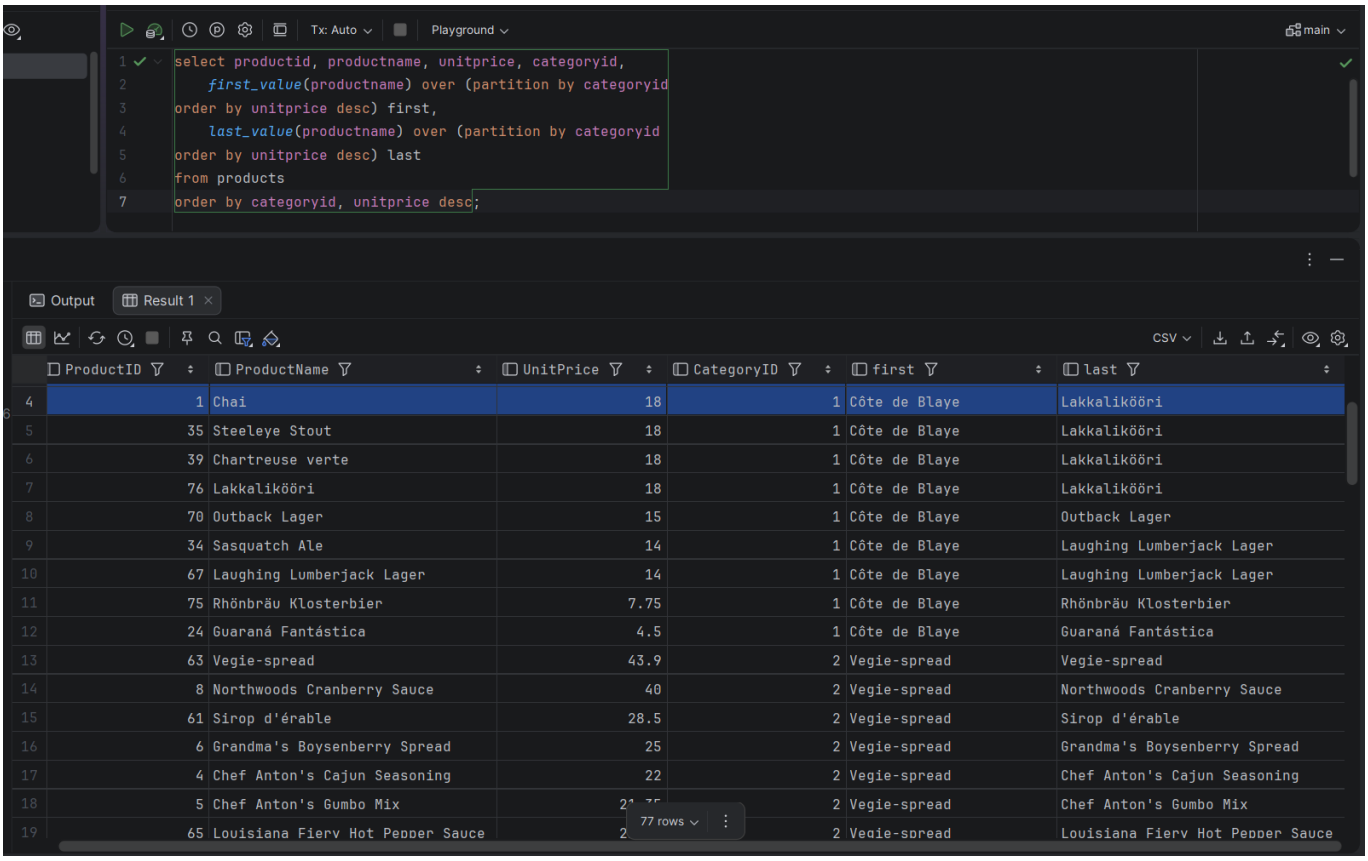
Co jest przyczyną takiego działania funkcji `last_value`.

Co trzeba zmienić żeby funkcja last_value pokazywała najtańszy produkt w danej kategorii?

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite

```
select productid, productname, unitprice, categoryid,
    first_value(productname) over (partition by categoryid
order by unitprice desc) first,
    last_value(productname) over (partition by categoryid
order by unitprice desc) last
from products
order by categoryid, unitprice desc;
```

Wyniki:



```
select productid, productname, unitprice, categoryid,
    first_value(productname) over (partition by categoryid
order by unitprice desc) first,
    last_value(productname) over (partition by categoryid
order by unitprice desc) last
from products
order by categoryid, unitprice desc;
```

	ProductID	ProductName	UnitPrice	CategoryID	first	last
4	1	Chai	18	1	Côte de Blaye	Lakkalikööri
5	35	Steeleye Stout	18	1	Côte de Blaye	Lakkalikööri
6	39	Chartreuse verte	18	1	Côte de Blaye	Lakkalikööri
7	76	Lakkalikööri	18	1	Côte de Blaye	Lakkalikööri
8	70	Outback Lager	15	1	Côte de Blaye	Outback Lager
9	34	Sasquatch Ale	14	1	Côte de Blaye	Laughing Lumberjack Lager
10	67	Laughing Lumberjack Lager	14	1	Côte de Blaye	Laughing Lumberjack Lager
11	75	Rhönbräu Klosterbier	7.75	1	Côte de Blaye	Rhönbräu Klosterbier
12	24	Guaraná Fantástica	4.5	1	Côte de Blaye	Guaraná Fantástica
13	63	Vegie-spread	43.9	2	Vegie-spread	Vegie-spread
14	8	Northwoods Cranberry Sauce	40	2	Vegie-spread	Northwoods Cranberry Sauce
15	61	Sirop d'érable	28.5	2	Vegie-spread	Sirop d'érable
16	6	Grandma's Boysenberry Spread	25	2	Vegie-spread	Grandma's Boysenberry Spread
17	4	Chef Anton's Cajun Seasoning	22	2	Vegie-spread	Chef Anton's Cajun Seasoning
18	5	Chef Anton's Gumbo Mix	21.35	2	Vegie-spread	Chef Anton's Gumbo Mix
19	65	Louisiana Fiery Hot Pepper Sauce	2	2	Vegie-spread	Louisiana Fiery Hot Pepper Sauce

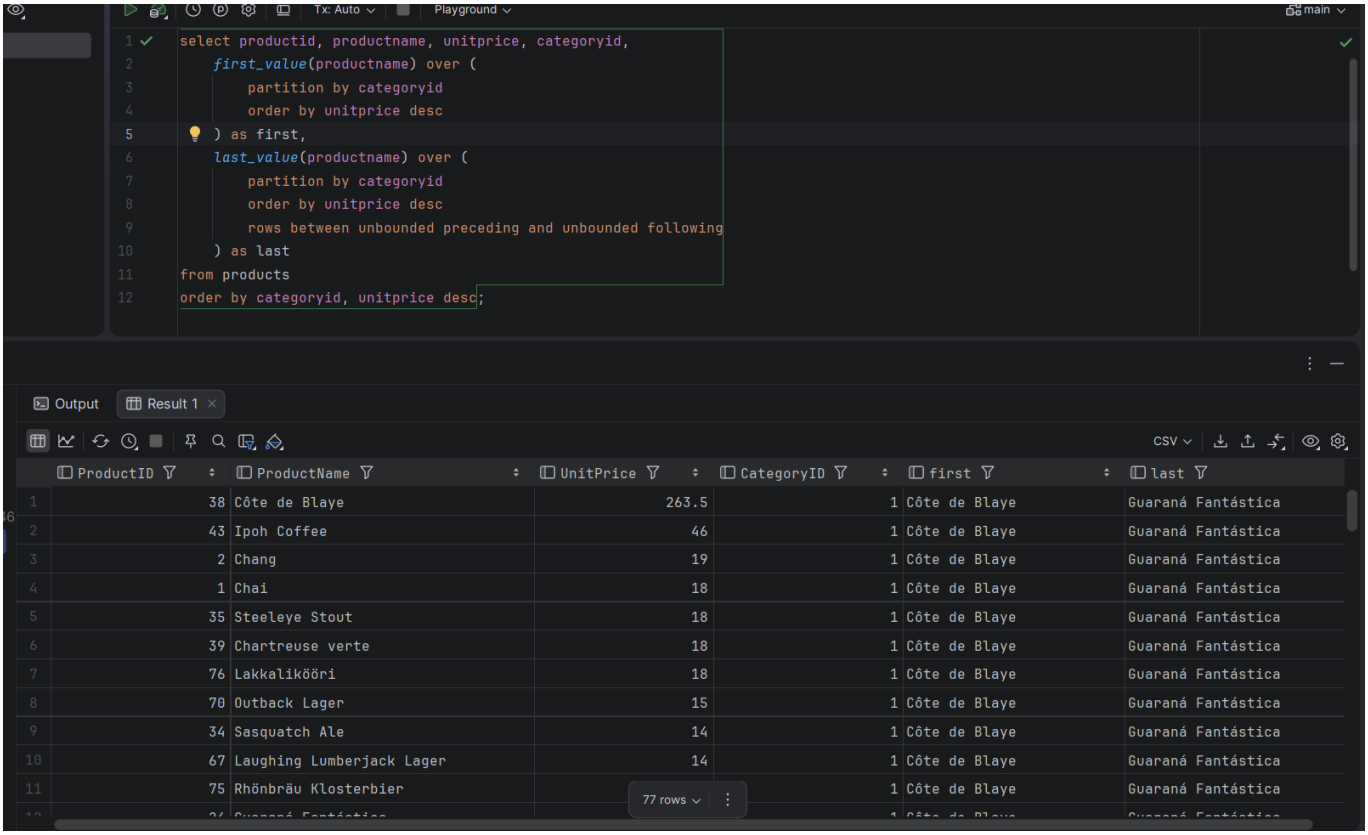
`first_value` bierze wartość z pierwszego wiersza w danym oknie, czyli zwraca najdroższy produkt w danej kategorii.
`last_value` bierze wartość z ostatniego wiersza w danym oknie.
W przypadku funkcji `last_value` - okno jest definiowane jako "od początku do aktualnego wiersza" Dlatego funkcja `last_value` nie pokazuje najtańszego produktu, tylko pokazuje produkt z aktualnego wiersza, czyli najtańszy produkt w danej chwili, bez uwzględnienia wierszy, które są po nim.

Aby funkcja `last_value` pokazywała najtańszy produkt w danej kategorii, trzeba rozszerzyć okno, tak aby obejmowało wszystkie wiersze w danej kategorii. Można to zrobić za pomocą klauzuli `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`

Poprawione zapytanie:

```
select productid, productname, unitprice, categoryid,
    first_value(productname) over (
        partition by categoryid
        order by unitprice desc
    ) as first,
    last_value(productname) over (
        partition by categoryid
        order by unitprice desc
        rows between unbounded preceding and unbounded following
    ) as last
from products
order by categoryid, unitprice desc;
```

Wynik zapytania z poprawionym last_value:



```
select productid, productname, unitprice, categoryid,
    first_value(productname) over (
        partition by categoryid
        order by unitprice desc
    ) as first,
    last_value(productname) over (
        partition by categoryid
        order by unitprice desc
        rows between unbounded preceding and unbounded following
    ) as last
from products
order by categoryid, unitprice desc;
```

	ProductID	ProductName	UnitPrice	CategoryID	first	last
1	38	Côte de Blaye	263.5	1	Côte de Blaye	Guaraná Fantástica
2	43	Ipoh Coffee	46	1	Côte de Blaye	Guaraná Fantástica
3	2	Chang	19	1	Côte de Blaye	Guaraná Fantástica
4	1	Chai	18	1	Côte de Blaye	Guaraná Fantástica
5	35	Steeleye Stout	18	1	Côte de Blaye	Guaraná Fantástica
6	39	Chartreuse verte	18	1	Côte de Blaye	Guaraná Fantástica
7	76	Lakkalikööri	18	1	Côte de Blaye	Guaraná Fantástica
8	70	Outback Lager	15	1	Côte de Blaye	Guaraná Fantástica
9	34	Sasquatch Ale	14	1	Côte de Blaye	Guaraná Fantástica
10	67	Laughing Lumberjack Lager	14	1	Côte de Blaye	Guaraná Fantástica
11	75	Rhönbräu Klosterbier		1	Côte de Blaye	Guaraná Fantástica

Zadanie 6

Baza: Northwind, tabele orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- Id klienta,
- nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- dane zamówienia klienta o najniższej wartości w danym miesiącu
    - nr zamówienia o najniższej wartości w danym miesiącu
    - datę tego zamówienia
    - wartość tego zamówienia
- dane zamówienia klienta o najwyższej wartości w danym miesiącu
    - nr zamówienia o najniższej wartości w danym miesiącu
    - datę tego zamówienia
    - wartość tego zamówienia

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite

Wyniki:

```sql
with order_values as (
    select
        o.customerid,
        o.orderid,
        o.orderdate,
        sum(od.unitprice * od.quantity * (1 - od.discount)) + o.freight as
order_value,
        strftime('%Y-%m', o.orderdate) as year_month
    from orders o
    join "Order Details" od on o.orderid = od.orderid
    group by o.customerid, o.orderid, o.orderdate, o.freight
)

select
    *,

    -- MIN
    first_value(orderid) over (
        partition by year_month
        order by order_value asc
    ) as min_order_id,

    first_value(orderdate) over (
        partition by year_month
```

```sql
        order by order_value asc
    ) as min_order_date,

    first_value(order_value) over (
        partition by year_month
        order by order_value asc
    ) as min_order_value,

    -- MAX
    first_value(orderid) over (
        partition by year_month
        order by order_value desc
    ) as max_order_id,

    first_value(orderdate) over (
        partition by year_month
        order by order_value desc
    ) as max_order_date,

    first_value(order_value) over (
        partition by year_month
        order by order_value desc
    ) as max_order_value

from order_values;
```

```sql
with order_values as (
    select
        o.customerid,
        o.orderid,
        o.orderdate,
        sum(od.unitprice * od.quantity * (1 - od.discount)) + o.freight as order_value,
        strftime('%Y', o.orderdate) as year,
        strftime('%m', o.orderdate) as month
    from orders o
    join "Order Details" od 1<->1..n: on o.orderid = od.orderid
    group by o.customerid, o.orderid, o.orderdate, o.freight
),
```

Output | Result 2

customerid	orderid	orderdate	order_value	year	month	rn_min	rn_max	mi
SPLIR	10271	1996-01-08	52.54	1996	01	1	4	
ISLAT	10318	1996-01-10	245.13	1996	01	2	3	
WARTH	10270	1996-01-08	1512.54	1996	01	3	2	
WHITC	10344	1996-01-11	2319.29	1996	01	4	1	
VINET	10295	1996-02-09	122.75	1996	02	1	4	
TORTU	10319	1996-02-10	1255.7	1996	02	2	3	
RATTC	10272	1996-02-08	1554.03	1996	02	3	2	
SPLIR	10369	1996-02-12	2586.08	1996	02	4	1	
LAMAI	10371	1996-03-12	73.41	1996	03	1	5	
ISLAT	10321	1996-03-10	147.43	1996	03	2	4	
WARTH	10320	1996-03-10	550.57	1996	03	3	3	
LILAS	10296	1996-03-09			03	4	2	
CHOPS	10370	1996-03-12			03	5	1	

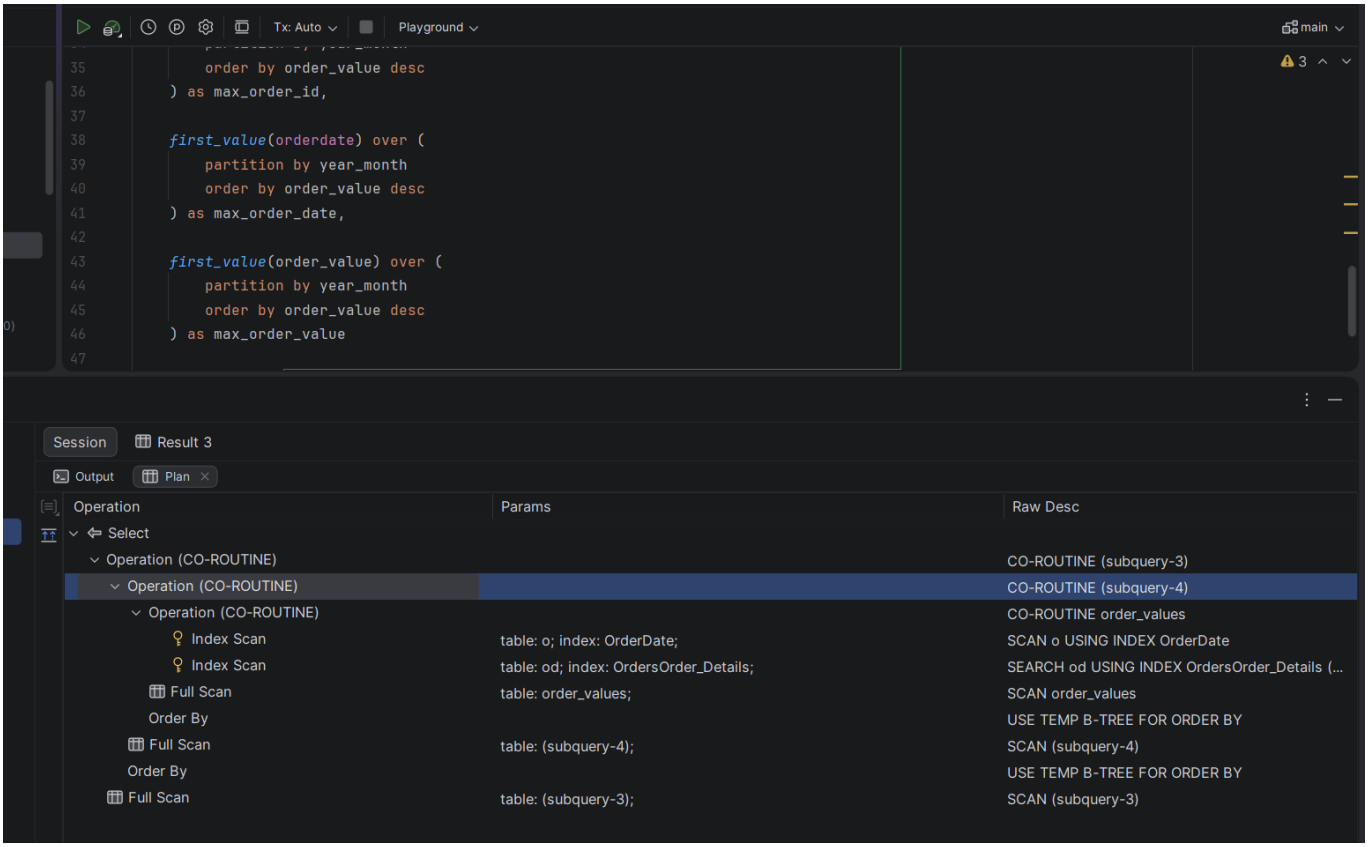
1-500 of 501+

Output | Result 2

	min_order_id	min_order_date	min_order_value	max_order_id	max_order_date	max_order_value
1	4	10271	1996-01-08	52.54	10344	1996-01-11	2319.29
2	3	10271	1996-01-08	52.54	10344	1996-01-11	2319.29
3	2	10271	1996-01-08	52.54	10344	1996-01-11	2319.29
4	1	10271	1996-01-08	52.54	10344	1996-01-11	2319.29
5	4	10295	1996-02-09	122.75	10369	1996-02-12	2586.08
6	3	10295	1996-02-09	122.75	10369	1996-02-12	2586.08
7	2	10295	1996-02-09	122.75	10369	1996-02-12	2586.08
8	1	10295	1996-02-09	122.75	10369	1996-02-12	2586.08
9	5	10371	1996-03-12	73.41	10370	1996-03-12	1118.77
10	4	10371	1996-03-12	73.41	10370	1996-03-12	1118.77
11	3	10371	1996-03-12	73.41	10370	1996-03-12	1118.77
12	2	10371	1996-03-12		10370	1996-03-12	1118.77
13	1	10371	1996-03-12		10370	1996-03-12	1118.77

1-500 of 501+

Plan zapytania dla SQLite:



Uzycie first value umożliwia wykonanie zadania bez użycia dodatkowych joinów, podzapytań czy agregacji.

Zadanie 7

Baza: Northwind, tabela product_history

Napisz polecenie które pokaże wartość sprzedaży każdego produktu narastająco od początku każdego miesiąca. Użyj funkcji okna

Zbiór wynikowy powinien zawierać:

- id pozycji
- id produktu
- datę
- wartość sprzedaży produktu w danym dniu
- wartość sprzedaży produktu narastające od początku miesiąca

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

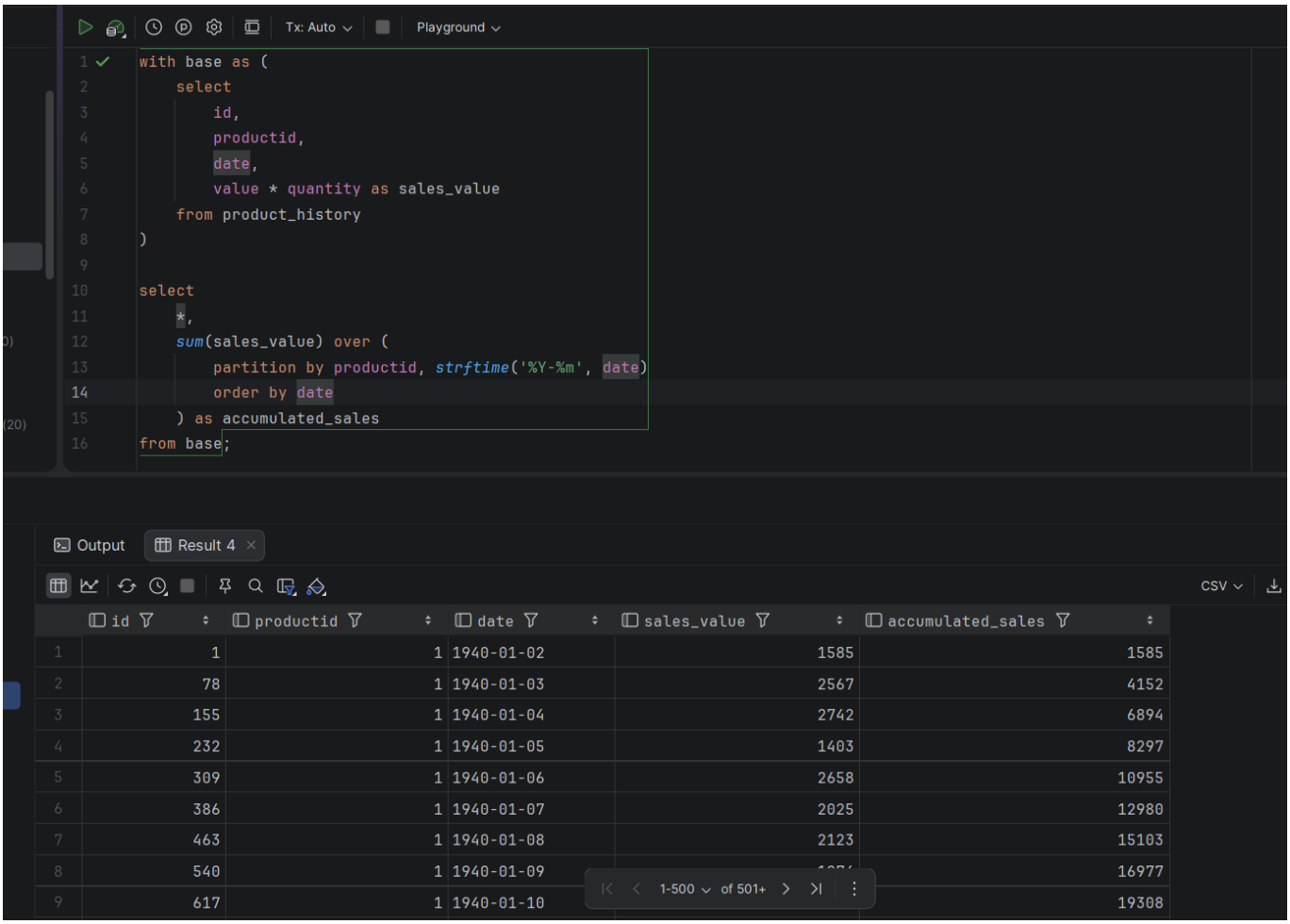
Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSql, SQLite)

Wyniki:

Kod SQL z funkcją okna dla SQLite:

```sql
with base as (
    select
        id,
        productid,
        date,
        value * quantity as sales_value
    from product_history
)

select
    *,
    sum(sales_value) over (
        partition by productid, strftime('%Y-%m', date)
        order by date
    ) as cumulative_sales
from base;
```

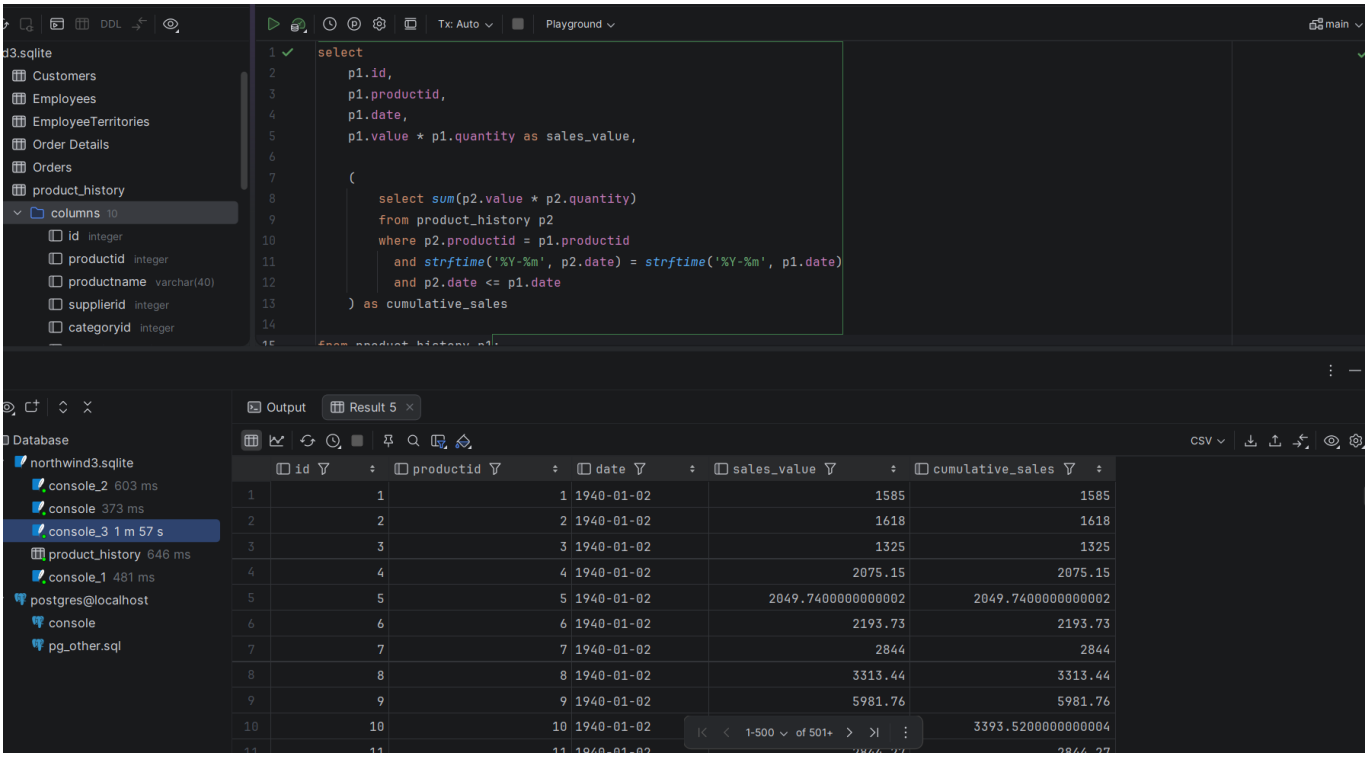
Rezultat wywołania powyższego zapytania:



Kod SQL bez funkcji okna dla SQLite:

```sql
select
    p1.id,
    p1.productid,
    p1.date,
    p1.value * p1.quantity as sales_value,

    (
        select sum(p2.value * p2.quantity)
        from product_history p2
        where p2.productid = p1.productid
          and strftime('%Y-%m', p2.date) = strftime('%Y-%m', p1.date)
          and p2.date <= p1.date
    ) as cumulative_sales

from product_history p1;
```

Rezultat wywołania powyższego zapytania:



Czasy zapytań z funkcją okna:

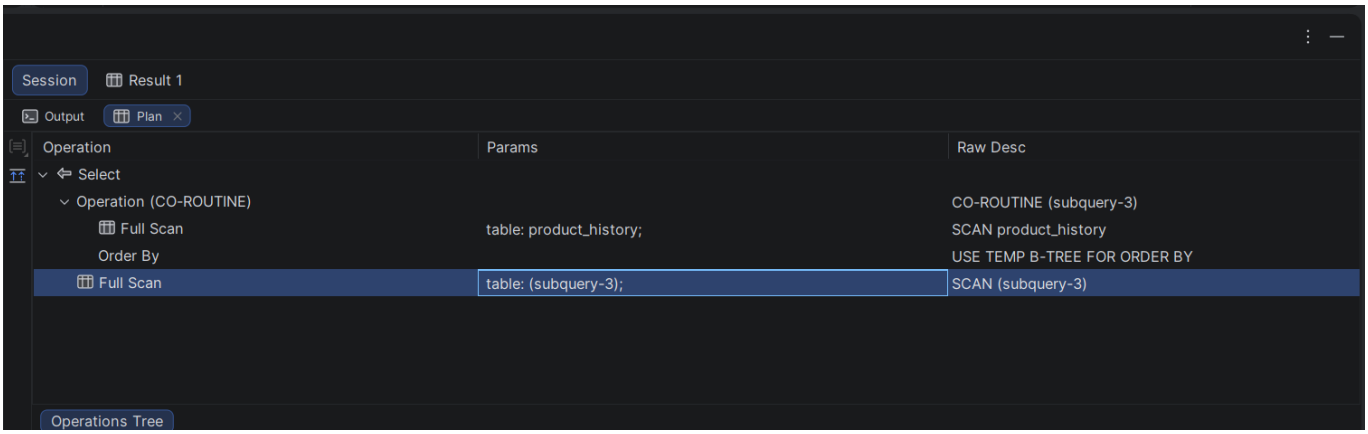
- MS SQL Server: 10 s
- SQLite: 2,4 s
- PostgreSQL: 1,5 s

Czasy zapytań bez funkcji okna:

- MS SQL Server: 34 s
- SQLite: 1 min 47 s
- PostgreSQL: 30 s

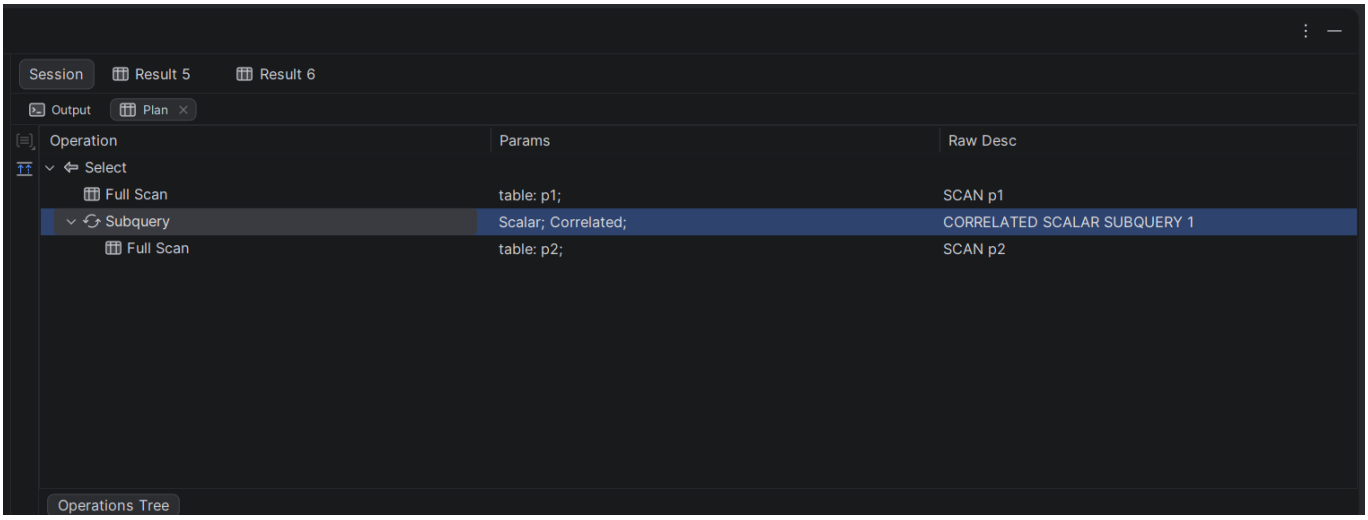
Plany dla zapytań z funkcją okna:

-SQLite:



Plany dla zapytań bez funkcji okna:

-SQLite:



Zadanie 8

Wykonaj kilka "własnych" przykładowych analiz.

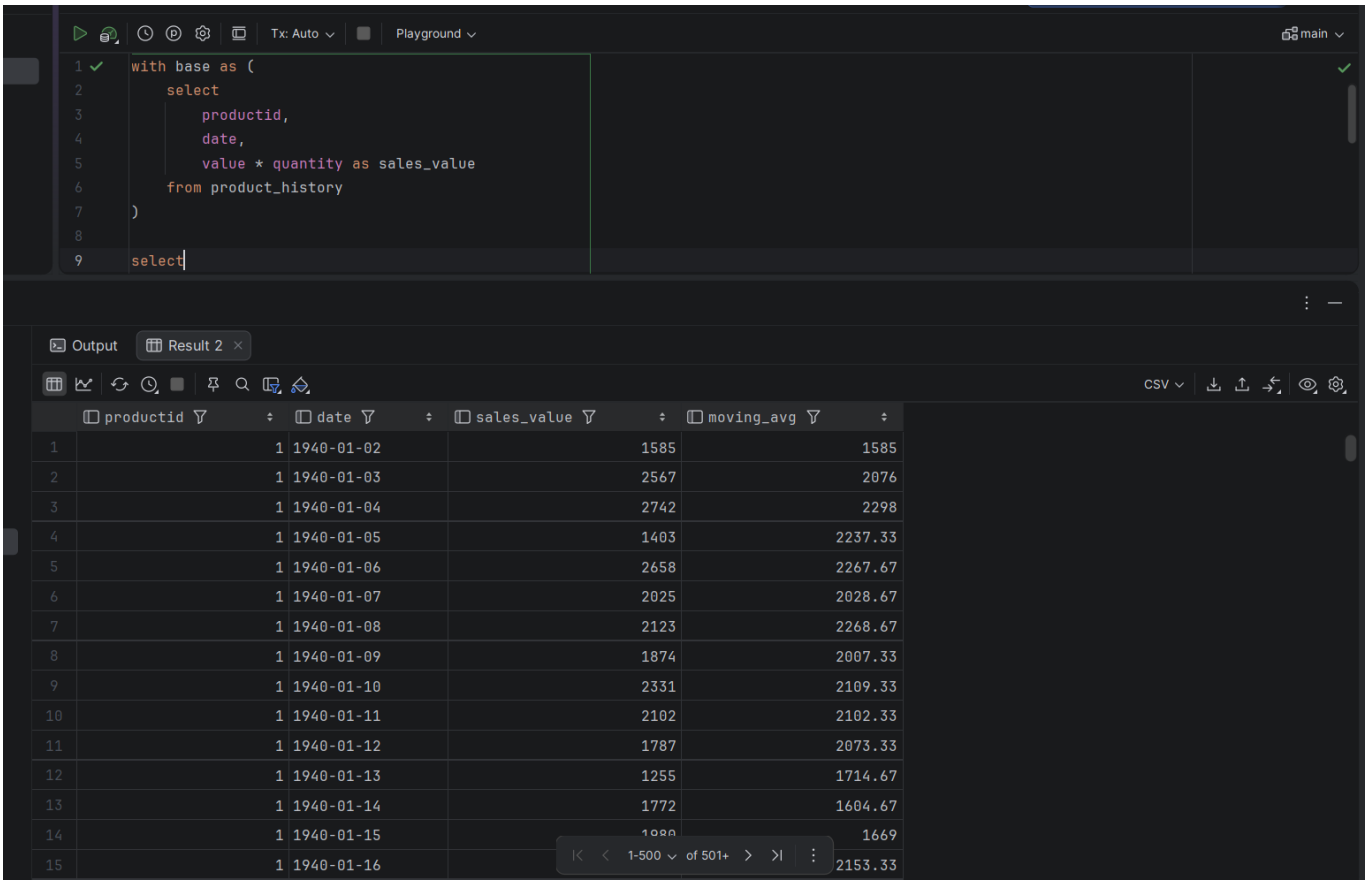
Czy są jeszcze jakieś ciekawe/przydatne funkcje okna (z których nie korzystałeś w ćwiczeniu)? Spróbuj ich użyć w zaprezentowanych przykładach.

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite

Średnia ruchoma (moving average) dla wartości sprzedaży produktu w danym dniu, z oknem 3 dniowym:

```sql
select
    productid,
    date,
    value * quantity as sales_value,

    avg(value * quantity) over (
        partition by productid
        order by date
        rows between 2 preceding and current row
    ) as moving_avg

from product_history;
```

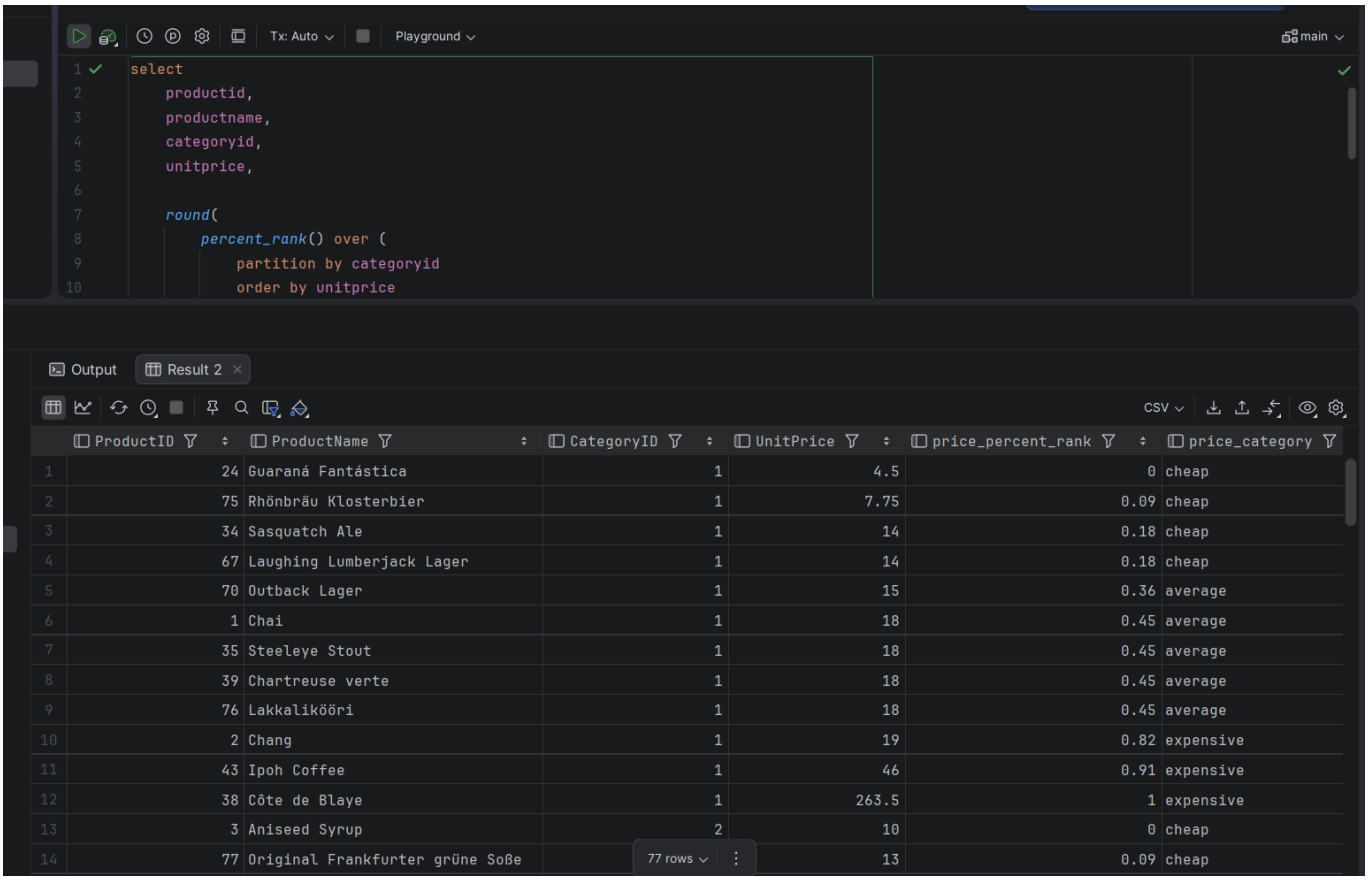
Rezultat wywołania powyższego zapytania:



Użycie funkcji percet_rank(), procentowa pozycja produktu w kategorii względem ceny

```
select
    productid,
    productname,
    categoryid,
    unitprice,

    percent_rank() over (
        partition by categoryid
        order by unitprice
    ) as price_percent_rank

from products;
```

Rezultat wywołania powyższego zapytania:



```
select
    productid,
    productname,
    categoryid,
    unitprice,

    round(
        percent_rank() over (
            partition by categoryid
            order by unitprice
```

Output | Result 2

	ProductID	ProductName	CategoryID	UnitPrice	price_percent_rank	price_category
1	24	Guaraná Fantástica	1	4.5	0	cheap
2	75	Rhönbräu Klosterbier	1	7.75	0.09	cheap
3	34	Sasquatch Ale	1	14	0.18	cheap
4	67	Laughing Lumberjack Lager	1	14	0.18	cheap
5	70	Outback Lager	1	15	0.36	average
6	1	Chai	1	18	0.45	average
7	35	Steeleye Stout	1	18	0.45	average
8	39	Chartreuse verte	1	18	0.45	average
9	76	Lakkalikööri	1	18	0.45	average
10	2	Chang	1	19	0.82	expensive
11	43	Ipoh Coffee	1	46	0.91	expensive
12	38	Côte de Blaye	1	263.5	1	expensive
13	3	Aniseed Syrup	2	10	0	cheap
14	77	Original Frankfurter grüne Soße		13	0.09	cheap

77 rows

Ranking klientów na podstawie łącznej wartości zamówień, z podziałem na kwartyle z wykorzystaniem funkcji ntile():

```sql
WITH CustomerSpending AS (
    SELECT
        o.CustomerID,
        SUM(od.UnitPrice * od.Quantity) as TotalSpent
    FROM Orders o
    JOIN [Order Details] od ON o.OrderID = od.OrderID
    GROUP BY o.CustomerID
)
SELECT
    CustomerID,
    TotalSpent,
    NTILE(4) OVER(ORDER BY TotalSpent DESC) as SpendingQuartile
FROM CustomerSpending;
```

Rezultat wywołania powyższego zapytania:

```sql
WITH CustomerSpending AS (
    SELECT
        o.CustomerID,
        SUM(od.UnitPrice * od.Quantity) as TotalSpent
    FROM Orders o
    JOIN [Order Details] od ON o.OrderID = od.OrderID
    GROUP BY o.CustomerID
)
SELECT
    CustomerID,
    TotalSpent,
    NTILE(4) OVER(ORDER BY TotalSpent DESC) as SpendingQuartile
```

Output | Result 5

CustomerID	TotalSpent	SpendingQuartile
GREAL	19711.13	1
BLONP	19088	1
SIMOB	18138.45	1
LINOD	17889.55	2
LILAS	17825.06	2
SEVES	17172.05	2
VAFFE	16643.8	2
WARTH	16617.1	2
OLDWO	16325.15	2
EASTC	15033.66	2
AROUT	13806.5	2
OTTIK	13157.5	2
RICAR	12924.4	2

89 rows

Punktacja

zadanie	pkt
1	1
2	2
3	1
4	1
5	1
6	1
7	2
8	2
razem	11